

Non traditional algorithms:-

- Genetic algorithms → theory of natural evolution
- Simulated annealing → Annealing process of material heat treatment.
- Particle swarm optimization → Movement of swarm of fishes or flying of a flocks of birds.
- Ant-Colony optimization → food searching techniques of a colony of ants.

Features :-

- i) A population of points is used instead of a single design point.
- ii) Since several points are used as candidate solutions, less likely to get trapped at a local optimum, gives global optimum.
- iii) Use only the values of the objective function. The derivatives are not used in the search procedure.
- iv) The design variables are represented as strings of binary variables and is applicable for solving discrete and integer programming problems.
- v) The objective function value corresponding to a design vector plays the role of fitness.

Genetic Algorithm :-

It is based on the principles of natural genetics and natural selection. The basic elements of natural genetics — reproduction, struggle for existence, Natural selection, crossovers and mutation — are used in the genetic search procedure.

Theory of natural evolution → Highly improved chromosomes in human being evolved from simplest and least qualified mono-cells animal. And following the rules of generation of new genes by crossover and mutation and preservation of good genes by natural selection (survival of the fittest)

Genetic operators $\rightarrow$ crossover, mutation, natural selection (random/probabilistic)

ALGORITHM 1-

The computational procedure involved in maximizing the fitness function $F(x_1, x_2, x_3, \dots, x_n)$ in the genetic algorithm can be described by the following steps—

1. Choose a suitable string length $l = nq$ to represent the n design variables of the design vector X . Assume suitable values for the following parameters—
 - population size $\rightarrow m$
 - crossover probability $\rightarrow p_c$
 - mutation probability $\rightarrow p_m$
 - permissible value of standard deviation of fitness values of the population $(S_f)_{\max}$ to use as a convergence criterion, ~~maximum~~ no. of generations $(i_{\max})$ to be used and a second convergence criterion.
2. Generate a random population of size m , each consisting of a string of length $l = nq$.
Evaluate the fitness values, $f_i \rightarrow i = 1, 2, \dots, m$ of the m strings.
3. Carry out the reproduction process.
4. Carry out the crossover operation using the crossover probability p_c .
5. Carry out the mutation operation using the mutation probability p_m to find the new generation of m strings.
6. Evaluate the fitness values $f_i \rightarrow i = 1, 2, 3, \dots, m$ of the m strings of the new population; find the standard deviation of the m fitness values.
7. Test for the convergence of the algorithm or process. If $S_f \leq (S_f)_{\max}$, the convergence criterion is satisfied and

hence the process may be stopped. Otherwise go to step 8.

8. Test for the generation number. If $i \geq i_{\max}$, the computations have been performed for the maximum permissible no. of generations and hence the process may be stopped. Otherwise, set the generation no. as $i+1$ and go to step 3.

Simulated Annealing

The simulated annealing method is based on the simulation of the thermal annealing of critically heated solids.

1. Start with an initial design vector X_1 (iteration no. $i=1$) and high value of temperature T .

2. Generate a new design point randomly in the vicinity of the current design point.

3. Find the difference in function values

$$\Delta E = \Delta f = f_{i+1} - f_i = f(X_{i+1}) - f(X_i)$$

4. If f_{i+1} is smaller than f_i (with a negative value of Δf), accept the point X_{i+1} as the next design point.

Otherwise if f_{i+1} is positive, accept the point X_{i+1} as the next design point only with a probability $e^{-\Delta E/KT}$. This means that if the value of a randomly generated no. is larger than $e^{-\Delta E/KT}$, accept the point X_{i+1} ; otherwise reject the point X_{i+1} .

This completes one iteration of the SA algorithm.

5. If the point X_{i+1} is rejected, then the process of generating a new design point X_{i+1} , randomly in the vicinity of the current design point,

6. Repeat steps 2 to 4.

To simulate the attainment of thermal equilibrium at every temperature, a predetermined no. (n) of new points X_{i+1} are tested at any specific value of the temperature T . Once the no. of new design points X_{i+1} tested at any T exceeds the value of n ,

7. Temperature T is reduced by a pre-specified fractional value α ($0 < \alpha < 1$) and the whole process is repeated several times.

8. The procedure is assumed to have converged when the current value of temperature T is sufficiently small or when changes in the function values (Δ)

are observed to be sufficiently small.

Features of the method

1. The quality of the final solution is not affected by the initial guesses, except that the computational effort may increase with worse starting designs.
2. Because of the discrete nature of the function and constraint evaluations the convergence or transition characteristics are not affected by the continuity or differentiability of the functions.
3. The convergence is also not influenced by the convexity status of the feasible space.
4. The design variables need not be positive.
5. The method can be used to solve mixed-integers, discrete or continuous problems.
6. For problems involving behavior constraints, an equivalent unconstrained function is to be formulated as in the case of genetic algorithms.

Particle Swarm Optimization

PSO is based on the behavior of a colony or swarm of insects such as ants, termites; a flock of birds; or a school of fish. It mimics the behavior of these social organisms. Optimization methods based on swarm intelligence are called behaviorally inspired algorithms as opposed to the genetic algorithms, which are called evolution-based procedures.

In the context of multivariable optimization, the swarm is assumed to be of specified or fixed size with each particle located initially at random locations in the multidimensional design space.

Each particle is assumed to have two characteristics —

- (a) a position,
- (b) a velocity.

Each particle wanders around in the design space and remembers the best position. The particles communicate information to each other and adjust their positions and velocities based on the information received at the good positions. Position of individuals are modified based on own experience and collective experience. Information obtained are used to find out best direction towards optimum.

The behavior of the swarm is based on a combination of 3 simple factors —

- (a) Cohesion — stick together.
- (b) Separation — Don't come too close.
- (c) Alignment — follow the general heading of the flock.

Consider an unconstrained maximization problem +

Maximize $f(X)$ with $X_L \leq X \leq X_U$

where X_L and X_U denotes lower and upper bounds on X respectively.

1. Assume, the size of the swarm $\rightarrow N$
usually, a size of 20-30 particles is assumed for the swarm as a compromise.

2. Generate the initial population of X in the range X_L and X_U randomly as $X_1, X_2, \dots, X_N$.

i th particle and its velocity in i th iteration $\rightarrow X_i(i)$

Thus the particles generated initially are $\rightarrow X_1(0), X_2(0), \dots, X_N(0)$.

The vectors $X_i(0)$ ($i=1, 2, \dots, N$) are called particles or vectors of coordinates of particles. Each member is a design solution consists of

N no. of design variables.

3. Find the velocities of particles. All particles will be moving to the optimal point with a velocity. Initially, all particle velocities $\rightarrow 0$.

Set iteration no. as $i=1$,

4. In the i th iteration, following 2 important parameters are available for a typical particle j for improvement in position —

(i) The historical best value of the individual $X_j(i), P_{best,j}$ with the highest value of the objective function $f[X_j(i)]$ encountered by particle j in all previous iterations.

All the N particles have such individual $P_{best,j}$

Best among all the $P_{best,j}$ is the global best G_{best}

The $X_j(i)$ corresponding to G_{best} , with the highest value of the objective function $f[X_j(i)]$, encountered in all previous iterations by any of the N particles.

Displacement towards these points will improve the objective function value. Velocity of a particle should have components along these two directions $(P_{best,j} - X_{j,i-1})$ and $(G_{best} - X_{j,i-1})$.
Velocity of a particle j in the i th iteration is defined as

$$V_j(i) = V_j(i-1) + c_1 r_1 [P_{best,j} - X_j(i-1)] + c_2 r_2 [G_{best} - X_j(i-1)] ; j=1, 2, \dots, N$$

where c_1 and c_2 are the individual and group learning rates and r_1 and r_2 are uniformly distributed random numbers in the range 0 and 1.

The values of c_1 and c_2 are usually assumed to be 2 so that $c_1 r_1$ and $c_2 r_2$ ensure that the particles would overfly the target about half the time.

The position or coordinate of the j th particle in i th iteration is updated as —

$$X_j(i) = X_j(i-1) + V_j(i) ; j=1, 2, \dots, N.$$

Convergence Criteria →

- Ⓐ If the positions of all particles converge to the same set of values,
- Ⓑ If not satisfied, step 4 is repeated by updating the iteration no, $i = i+1$ and by computing the new values of $P_{best,j}$ and G_{best} .
- Ⓒ The iterative process is continued until all particles converge to the same optimum solution or after a fixed no. of iterations.

Either in velocity or in position updation a part of the old status is preserved and rest is modified based on information regarding the local of optimum.

$$V_j(i) = \theta V_j(i-1) + r_1 [P_{best,j} - X_j(i-1)] + r_2 [G_{best} - X_j(i-1)]$$

$$j = 1, 2, \dots, N.$$

$$\theta(i) = \theta_{max} - \left(\frac{\theta_{max} - \theta_{min}}{i_{max}} \right) i$$

Thus in each iteration the population is updated using local best and global best.

The global best sol^n in each iteration is the best solution and after convergence, the last global best sol^n is accepted as optimum sol^n .

Ant Colony Optimization:-

Ant Colony optimization (ACO) is based on the cooperative behaviour of real ant colonies, which are able to find the shortest path from their nest to a food source.

The step-by-step procedure of ACO algorithm for solving a minimization problem can be summarized as follows -

Step-1:- Assume a suitable no. of ants in the colony (N). Assume a set of permissible discrete values for each of the n design variables. Denote the permissible discrete values of the design variable x_i as $x_{i1}, x_{i2}, \dots, x_{ip}$ ($i=1, 2, \dots, n$).

Assume equal amounts of pheromone $\tau_{ij}(1)$ initially along all the arcs or rays of the multi-layered graph. The superscript in τ_{ij} is the iteration no.,

$\tau_{ij}(1) = 1$ can be assumed for all arcs ij . Set the iteration no. $l=1$.

Step-2:- (a) Compute the probability (P_{ij}) of selecting the arc or ray (on the discrete value) x_{ij} as

$$P_{ij} = \frac{\tau_{ij}(l)}{\sum_{m=1}^P \tau_{im}(l)} ; i=1, 2, \dots, n ; j=1, 2, \dots, P$$

which can be same as $\Rightarrow$ An ant k , when located at node i uses the pheromone trail τ_{ij} to compute the probability of choosing j as the next node with $\alpha=1$

$$P_{ij}(k) = \begin{cases} \frac{\tau_{ij}^\alpha}{\sum_{j \in N_i(k)} \tau_{ij}^\alpha} & \text{if } j \in N_i(k) \\ 0 & \text{if } j \notin N_i(k) \end{cases}$$

(b) The specific path chosen by the k th ant can be determined using random numbers generated in the range $(0, 1)$. For this, we find the cumulative probability ranges associated with different paths. The specific path chosen by ant k will be determined using the roulette-wheel selection process.

Step 3 +

- (a) Generate N random numbers $r_1, r_2, \dots, r_N$ in the range $(0, 1)$, one for each ant. Determine the discrete value path assumed by ant k for variable i as the one for which the cumulative probability range includes the value r_i .
- (b) Repeat 3(a) for all design variables, $i = 1, 2, \dots, n$.
- (c) Evaluate the objective function values corresponding to the complete paths

$$f_k = f(x(k)) \quad k = 1, 2, \dots, N$$

Determine the best and worst paths among the N paths chosen by different ants +

$$f_{\text{best}} = \min_{k=1,2,\dots,N} \{f_k\}$$

$$f_{\text{worst}} = \max_{k=1,2,\dots,N} \{f_k\}$$

Step 4 + Test for the convergence of the process. The process is assumed to have converged if all N ants take the same best path. If convergence is not achieved, assume that all the ants return home and start again in search of food, let the iteration no. as $l = l + 1$ and update the pheromones on different arcs as

$$\tau_{ij}(l) = \tau_{ij}(\text{old}) + \sum_k \Delta \tau_{ij}(k)$$

where $\tau_{ij}(\text{old})$ denotes the pheromone amount of the previous iteration left after evaporation, which is taken as —

$$\tau_{ij}(\text{old}) = (1 - \rho) \tau_{ij}^{(l-1)}$$

$\Delta \tau_{ij}(k)$ is the pheromone deposited by the best ant k on its path and the summation extends over all the best ants k . The evaporation rate on pheromone decay factor ρ is assumed to be in range 0.5 to 0.8 and with new values of $\tau_{ij}(l)$, goto step 2. Step 2, 3 and 4 are repeated until the process converges, that is all the ants chose the best path. In some cases, the iterative process is stopped after completing a prespecified max no. of iterations (l_{max})

General Form of Multi-Objective Optimization Problem:-

Mathematically,

$$\min/\max f_m(x), m=1, 2, \dots, M$$

$$\text{subject to } g_j(x) \geq 0, j=1, 2, \dots, J$$

$$h_k(x) = 0, k=1, 2, \dots, K$$

$$x_i(l) \leq x_i \leq x_i(u), i=1, 2, \dots, n.$$

Dominance:-

In multi-objective optimization problem, the goodness of a solⁿ is determined by the dominance.

Dominance Test $\Rightarrow x_1$ dominates x_2 , if —

(i) Soluⁿ x_1 is no worse than x_2 in all objectives.

(ii) Soluⁿ x_1 is strictly better than x_2 in at least one objective.

x_1 dominates $x_2 \Leftrightarrow x_2$ is dominated by x_1

Pareto-Optimal Soluⁿ:-

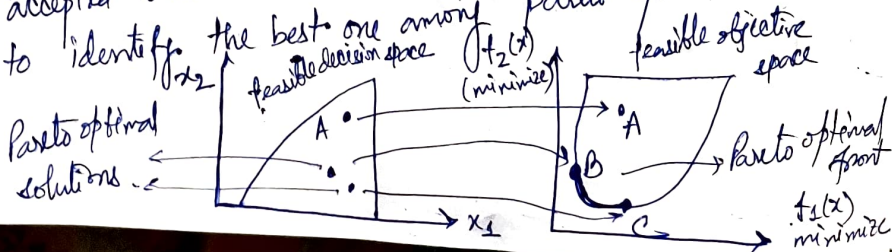
Given a set of solutions, the non-dominated solⁿ set is a set of all the solutions that are not dominated by any member of the solⁿ set.

The non-dominated set of the entire feasible decision space is called the Pareto-optimal set.

The boundary defined by the set of all point mapped from the Pareto optimal set is called the Pareto optimal front.

Any solⁿ on Pareto front is an optimum solⁿ having different degree of bias to different objective functions. None of the solutions can be accepted to be the best one among all. A new condition is essential to identify the best one among Pareto-optimal solutions.

Graphical depiction of Pareto optimal solution



① Dominated solⁿ $\rightarrow$ set of design points performing worse than some other better points.

Domination criterion $\rightarrow$ A feasible solⁿ x_1 dominates an other feasible solⁿ x_2 ($x_1 < x_2$), if both of the following conditions are true —

- ① The solⁿ x_1 is no worse than x_2 in all objectives, i.e. $f_i(x_1) \leq f_i(x_2)$
- ② The solⁿ x_1 is strictly better than x_2 in at least one objective i.e. $f_i(x_1) < f_i(x_2)$

② Non-dominated solⁿ $\rightarrow$ If two solⁿs are compared, then the solⁿs are said to be non-dominated w.r.t. each other if neither solⁿ dominates the other.

Pareto optimal front $\rightarrow$ the funcⁿ space representation of all the non-dominated solutions.

Solution Methods

Methods that try to avoid generating the Pareto front

- $\rightarrow$ Generate "utopia point"
- $\rightarrow$ Define optimum based on some measure of distance from utopia-point.

Generating entire Pareto front

— Weighted sum of objectives with variable coefficients.

— Optimize one objective for a range of constraints on the others.

— Niching methods with population based algorithms.

Weighted Sum Method

Scalarize a set of objectives into a single objective by adding each objective premultiplied by a user-supplied weight.

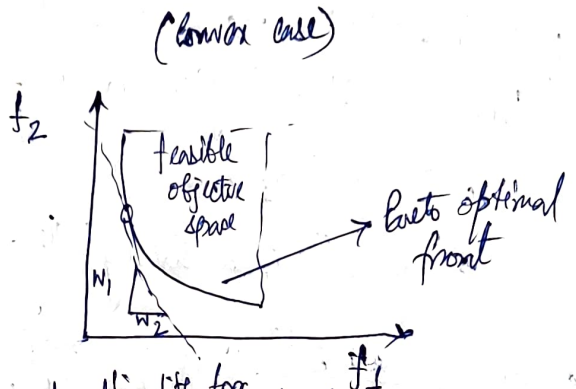
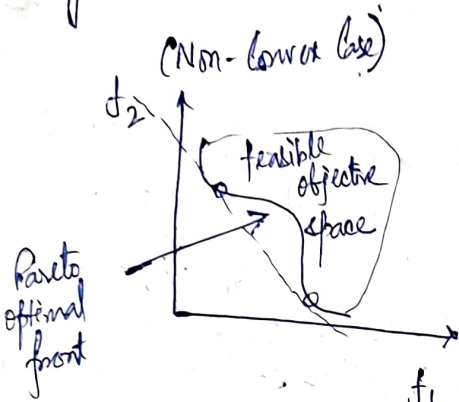
$$\text{minimize } F(x) = \sum_{m=1}^M w_m f_m(x),$$

$$\text{subject to } g_j(x) \geq 0, \quad j=1, 2, \dots, J$$

$$h_k(x) = 0, \quad k=1, 2, \dots, K$$

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i=1, 2, \dots, n$$

Weight of an objective is chosen in proportion to relative importance of the objective.



Advantage $\rightarrow$ Simple, guarantees Pareto optimality for convex objective spaces

Disadvantage $\rightarrow$ (i) It is difficult to set the weight vector to obtain a Pareto-optimal solution in a desired region in the objective space.

(ii) It cannot find certain Pareto-optimal solⁿs in the case of a nonconvex objective space.

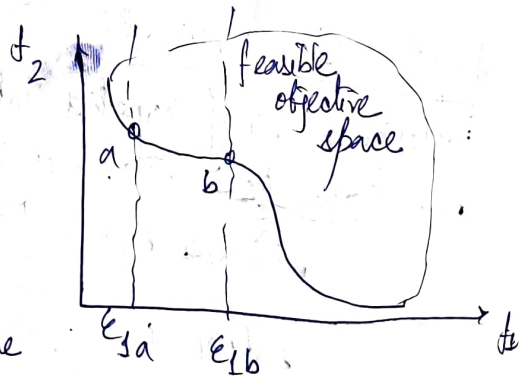
ϵ -constraint Method - Haines et. al. 1971

Keep just one of the objective and restricting the rest of the objectives within user-specific values.

$$\begin{aligned} &\text{minimize } f_1(x), \\ &\text{subject to } f_m(x) \leq \epsilon_m, \quad m=1, 2, \dots, M \text{ and } m \neq 1 \\ &\quad g_j(x) \geq 0, \quad j=1, 2, \dots, J \\ &\quad h_k(x) = 0, \quad k=1, 2, \dots, K \\ &\quad x_i(L) \leq x_i \leq x_i(U), \quad i=1, 2, \dots, n \end{aligned}$$

Advantage $\rightarrow$ Applicable to either convex or non-convex problems.

Disadvantage $\rightarrow$ The ϵ vector has to be chosen carefully so that it is within the minimum or maximum values of the individual objective funcⁿs.



Keep f_2 as an objective
minimize $f_2(x)$ treat f_1 as a
constraint $f_1(x) \leq \epsilon_1$

Weighted Metric Method

Combine multiple objectives using the weighted distance metric of any solⁿ from the ideal solⁿ z^*

$$\text{minimize } l_p(x) = \left(\sum_{m=1}^M w_m |f_m(x) - z_m^*|^p \right)^{1/p},$$

$$\text{subject to } g_j(x) \geq 0, \quad j = 1, 2, \dots, J$$

$$h_k(x) \leq 0, \quad k = 1, 2, \dots, K$$

$$x_i^{(L)} \leq x_i \leq x_i^{(U)}, \quad i = 1, 2, \dots, n$$

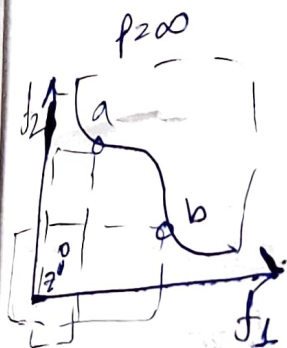
Advantage → Weighted Tchebycheff metric guarantees finding all Pareto optimal solⁿ with ideal solⁿ z^* .

Disadvantage → (a) Requires knowledge of minimum and maximum objective values.

(b) Requires z^* which can be found by independently optimizing each objective funcⁿs.

(c) For small p , not all Pareto-optimal solⁿs are obtained.

(d) As p increases, the problem becomes non-differentiable.



(Weighted Tchebycheff Problem)

Artificial Neural Networks:-

ANNs are software or hardware systems designed to simulate the operation of a simple biological nervous system.

Components of ANN —

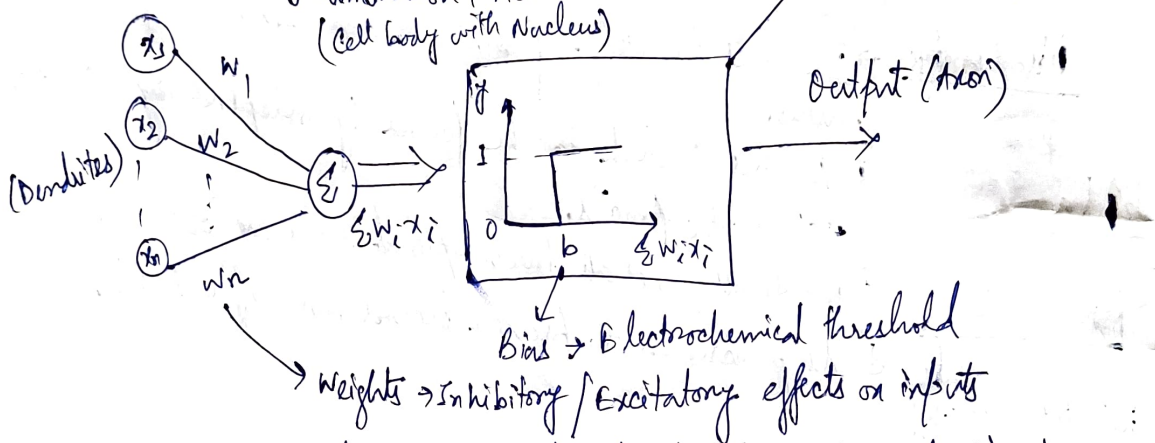
(i) Artificial Neurons

(ii) Topology — the connectivity path between neurons

(iii) Weights

(iv) A learning Algorithm

Summation + Activation function
(Cell body with Nucleus)



Similarly to biological brains, neural networks have the capacity to learn, memorize and create relationships amongst data. Their functionality depends on the neuron weights and on the network architecture.

Feedforward ANN are the most widely used type of ANN. Signals flow from the inputs to the outputs of the network through several layers of neurons. No feedback loops are present.

ANNs learn based on examples. Learning implies changes of neuron weights and biases.

Learning Methods — (i) Supervised learning

(ii) Unsupervised learning

(iii) Reinforcement learning

Design Problems +

- (i) Learning algorithms are not guaranteed to succeed.
- (ii) There are no general methods to determine the optimal no. of neurons necessary for solving any problem.
- (iii) It is difficult to select a training dataset which fully describes the problem to be solved.

Drawbacks +

- (i) The operation of large neural networks is impossible to analyze because of the huge no. of variables involved.
- (ii) It is often impossible to understand what the neural network "really thinks" even when a correct answer is generated.
- (iii) Neural systems need further research before they are widely accepted in industry.
- (iv) Current neuron models are simplistic compared to their biological counterparts.

Each artificial neuron computes a weighted sum of its inputs and a bias, $z = \sum_{i=1}^n w_i x_i + b$, then applies an activation function $f(z)$ (sigmoid/ReLU) to produce its output $y = f(z)$. The weights w_i and bias b control how strongly each input influences the neuron, analogous to synaptic strengths in biology.

In feed-forward ANNs, information passes strictly from input to output without cycles; this architecture is widely used for classification and regression tasks. Recurrent ANNs include feedback loops, enabling dynamic temporal behavior, useful in sequence modeling.

ANNs learn by adjusting weights to minimize a cost function (mean squared error) over training examples. The backpropagation algorithm computes gradients of the loss with respect to each weight via the chain rule, then updates weights using gradient descent.

$$w_i \leftarrow w_i - \eta \frac{\partial \mathcal{L}}{\partial w_i} \quad \text{where } \eta \text{ is learning rate.}$$

Non-linear ^{activation} functions enable the network to learn complex, non-linear mappings. Modern practice favors ReLU and its variants for faster convergence and mitigation of vanishing-gradient issues. Training typically proceeds in epochs, with data presented in batches and weights updated iteratively to minimize training loss and generalize to new data.

Kung's et. al. Method for identifying non-dominated solutions:-
Step 1 - Sort population in descending order of importance of the first objective function and name population as P .

Step 2 - Call recursive function $\text{front}(P)$

$\text{front}(P)$

if $|P| = 1$,
Return P

Else

$T = \text{front}(P(1 : \lfloor |P|/2 \rfloor))$

$B = \text{front}(P(\lfloor |P|/2 \rfloor + 1 : |P|))$

if the i -th non-dominated solution of B is not dominated
by any non-dominated solution of T , $M = T \cup \{i\}$

Return M

End

End

$|P| \Rightarrow$ no. of the elements

$P(a:b) \Rightarrow$ all the elements of P from index a to b

$\lfloor \cdot \rfloor$ is an operator gives the nearest smaller integer value